

Raport de Laborator: Optimizarea Sistemelor de Baze de Date

Set de date: 30.000 Înregistrări | **Stivă Tehnologică:** Spring Boot 3.3, Hibernate 6, Caffeine, SQLite

1. Rezumat Executiv

Acest raport analizează o serie de teste de performanță (benchmarks) efectuate pe o bază de date gestionată prin Spring Boot. Prin implementarea unor tehnici de optimizare la nivel de interogare, indexare, paginare, cache și procesare în masă, am obținut reduceri de latență cuprinse între 40% și 1500%. Aceste rezultate demonstrează că comportamentul standard al unui ORM este adesea insuficient pentru date la scară de producție fără o configurare specifică.

2. Metrici de Performanță și Analiză

I. Rezolvarea Problemei "N+1 Queries"

Tehnică: Eliminarea interogărilor iterative pentru entitățile copil prin JOIN FETCH.

- **Lazy Loading Standard:** 5.456,0 ms
- **Eager Fetching Optimizat:** 3.308,0 ms
- **Reducere Latență:** ~40%

Analiză: Interogarea standard rezultă în $N+1$ călătorii în rețea (1 pentru părinte, N pentru copii). Deși abordarea optimizată este mai rapidă, timpul de 3,3s pentru 30.000 de înregistrări indică faptul că blocajul principal s-a mutat de la SQL la "JVM Object Hydration" — timpul necesar pentru a instanția obiecte Java din seturile de rezultate brute.

II. Indexarea și Eficiența Căutării

Tehnică: Evaluarea impactului indicilor de tip B-Tree asupra diferitelor filtre de căutare.

- **Coloană Indexată (Email):** 69,5 ms
- **Cheie Externă Indexată (Departament):** 55,3 ms
- **Interval Neindexat (Salariu):** 2.938,8 ms
- **Index Compozit (Multi-coloană):** 37,7 ms

Analiză: Căutarea după salariu este de 78 de ori mai lentă decât căutarea multi-coloană. Fără un index, baza de date efectuează un "Full Table Scan". Un index compozit pe (department_id, salary) permite motorului să sară direct la rândurile relevante, obținând rezultate sub 50ms.

III. Paginarea: Offset vs. Keyset (Metoda Seek)

Tehnică: Compararea scanării prin OFFSET cu metoda indexată WHERE id > last_seen_id.

3. Compromisuri și Cazuri de Utilizare Strategice

Poziție	Paginare Offset (ms)	Paginare Keyset (ms)	Factor Viteză
Pagina 1	105,46 ms	5,94 ms	17x
Pagina Mijloc	35,52 ms	0,94 ms	37x
Ultima Pagină	28,74 ms	1,00 ms	28x

Analiză: Performanța paginării prin Offset este neregulată și în general lentă, deoarece baza de date trebuie să scaneze toate rândurile anterioare înainte de a le returna pe cele dorite. Paginarea prin Keyset oferă performanță de tip $SO(\log N)$, rămânând constantă la ~1 ms chiar și la adâncime maximă.

IV. Caching la Nivel de Aplicație (Caffeine)

Tehnică: Implementarea cache-ului local L2 cu TTL (Time To Live) și evacuare manuală.

- **Cache Miss (Preluare din DB):** 60,88 ms
- **Cache Hit (RAM):** 0,95 ms

Analiză: Caching-ul oferă o creștere a performanței de 60 de ori. Testele au demonstrat că evacuarea și expirarea TTL (testată la 11s) mențin eficient integritatea datelor, forțând reîmprospătarea după actualizări sau expirarea timpului.

V. Optimizarea Operațiunilor Bulk (În Masă)

Tehnică: Compararea metodei merge (individual) cu interogări native Bulk JPQL.

- **Abordarea 1 (Merge-uri individuale):** 512,48 ms
- **Abordarea 2 (Interogare Bulk JPQL):** 68,45 ms
- **Abordarea 3 (Batching Manual):** 400,95 ms

Analiză: Abordarea 2 este de 7,5 ori mai rapidă. Efectuând calculele direct în SQL, evităm suprasolicitarea Contextului de Persistență JPA și a mecanismului de "dirty checking".

VI. Reutilizarea Statement-urilor (Instrucțiunilor)

Tehnică: Reutilizarea obiectelor Query pentru a evita re-analizarea (parsing) JPQL.

- **Fără Reutilizare:** 589,98 ms
- **Cu Reutilizare:** 37,38 ms
- **Câștig:** De 15,78 ori mai rapid

Analiză: Acest punct evidențiază overhead-ul semnificativ pe care Hibernate îl suportă la traducerea JPQL în SQL. În bucle strânse (loops), reutilizarea este obligatorie pentru a menține utilizarea procesorului la un nivel scăzut.

Tehnică	Când se utilizează	Compromis
Indexare	Filtre de citire frecvente.	Încetinește operațiunile INSERT/UPDATE.
Caching	Date statice sau care se schimbă rar.	Consum ridicat de RAM; risc de date învechite.
Interogări Bulk	Actualizări de masă (ex: măririi salariale).	Ocolește ciclul de viață/listenerii JPA.
Paginare Keyset	UI-uri cu scroll infinit / flux de știri.	Nu permite saltul la un număr specific de pagină.

4. Listă de Verificare pentru Producție

- **Variabile de Mediu:** Toate șirurile de conexiune folosesc `${DB_PATH}` pentru a evita expunerea secretelor.
- **Connection Pooling:** HikariCP este configurat cu `maximum-pool-size=10` și `connection-test-query=SELECT 1`.
- **Scurgeri de Resurse:** Toate operațiunile EntityManager au loc în interiorul barierelor `@Transactional`.

- **Politică de Cache:** TTL și dimensiunea maximă sunt configurate pentru a preveni erorile de tip OutOfMemoryError.

5. Lecții Învățate și Recomandări

- **Lecția 1:** Scanările complete ale tabelelor ("Full Table Scans") sunt "ucigași tăcuți". O singură coloană neindexată într-o interogare de tip interval poate crește latența cu 3000%.
- **Lecția 2:** ORM-ul nu este magic. Deși JPA este convenabil, actualizările directe prin JPQL sunt necesare pentru performanța operațiunilor de masă.